

PostgreSQL Window Functions: Complete Guide

Target Audience: Anyone who knows basic SQL queries, `JOIN` s, and `GROUP BY` , and wants to master Window Functions from first principles to production-level data engineering.

Table of Contents

- 1. From `GROUP BY` to Window Functions (The "Aha!" Moment)
- 2. The Anatomy of `OVER()` : The Engine of Window Functions
- 3. Window Frames: `ROWS` vs `RANGE` (The Hidden Trap)
- 4. Category 1: Ranking Functions
 - 1. `ROW_NUMBER()`
 - 2. `RANK()`
 - 3. `DENSE_RANK()`
- 5. Category 2: Distribution Functions
 - 4. `PERCENT_RANK()`
 - 5. `CUME_DIST()`
 - 6. `NTILE()`
- 6. Category 3: Navigation / Offset Functions
 - 7. `LAG()`
 - 8. `LEAD()`
- 7. Category 4: Value Functions
 - 9. `FIRST_VALUE()`
 - 10. `LAST_VALUE()`
 - 11. `NTH_VALUE()`
- 8. Category 5: Common Aggregate-as-Window Functions
 - `SUM()` , `AVG()` , `COUNT()` , `MIN()` , `MAX()` with `OVER()`
- 9. SQL Execution Order: Why `WHERE` Doesn't Work Directly
- 10. Master Cheat Sheet

1. From `GROUP BY` to Window Functions

To understand window functions, let's start with what you already know: `GROUP BY` .

The Problem with `GROUP BY` : It Collapses Your Data

Suppose you have an `orders` table:

order_id	customer_id	total_amount
101	Cust_A	\$100

order_id	customer_id	total_amount
102	Cust_A	\$300
103	Cust_B	\$200
104	Cust_B	\$50

If you want to know the total spend per customer using `GROUP BY`:

```
SELECT customer_id, SUM(total_amount) AS customer_total
FROM orders
GROUP BY customer_id;
```

Output:

customer_id	customer_total
Cust_A	\$400
Cust_B	\$250

Notice what happened: **You lost your individual orders!**

You cannot see `order_id`, `101`, `$100`, or `$300` anymore. `GROUP BY` squashed 4 rows into 2 summary rows.

The Old, Painful Way (Before Window Functions)

What if business asks: *"Show every order, its amount, AND alongside it, show that customer's total spend and what percentage this order is of their total spend?"*

Without window functions, you had to write a subquery/CTE with a `GROUP BY`, and then **JOIN it back to the original table**:

```
-- The Painful Self-Join Way:
WITH customer_totals AS (
  SELECT customer_id, SUM(total_amount) AS total_spend
  FROM orders
  GROUP BY customer_id
)
SELECT
  o.order_id,
  o.customer_id,
  o.total_amount,
  c.total_spend,
  ROUND((o.total_amount / c.total_spend) * 100, 2) AS pct_of_customer_spend
FROM orders o
JOIN customer_totals c ON o.customer_id = c.customer_id;
```

Why is this bad?

1. It is verbose and hard to read.
2. The database has to scan the `orders` table twice, sort it, group it, and then join it. This is slow and expensive on large datasets.

The Window Function Way: Calculation Without Collapsing

A **Window Function** computes values across a group of rows (called a **window**), but **keeps every individual row intact**:

```
-- The Clean Window Function Way:
SELECT
  order_id,
  customer_id,
  total_amount,
  SUM(total_amount) OVER (PARTITION BY customer_id) AS total_spend,
  ROUND((total_amount / SUM(total_amount) OVER (PARTITION BY customer_id)) * 100, 2) AS pct_of_customer_spend
FROM orders;
```

Output:

order_id	customer_id	total_amount	total_spend	pct_of_customer_spend
101	Cust_A	\$100	\$400	25.00%
102	Cust_A	\$300	\$400	75.00%
103	Cust_B	\$200	\$250	80.00%
104	Cust_B	\$50	\$250	20.00%

Key Takeaway:

- GROUP BY **collapses** rows (4 rows in → 2 rows out).
- Window Functions **retain** all rows (4 rows in → 4 rows out), adding calculated columns alongside original data.

2. The Anatomy of OVER()

The keyword that transforms an ordinary function into a window function is **OVER()**.

FUNCTION(...) OVER (PARTITION BY ... ORDER BY ... ROWS/RANGE ...)

1. Slicing 2. Ordering 3. Framing

Let's break down each piece:

1. OVER () (Empty OVER clause)

Treats the **entire result set** as a single giant window.

```
-- Shows each order amount alongside the grand total of the entire store
SELECT order_id, total_amount,
       SUM(total_amount) OVER () AS grand_total
FROM orders;
```

2. PARTITION BY column_name

Divides the rows into separate buckets or groups (called **partitions**). The window calculation runs independently inside each bucket and resets when a new bucket starts.

Think of **PARTITION BY** as a **GROUP BY** that does not collapse rows.

3. ORDER BY column_name [ASC|DESC]

Sorts rows **inside each partition** so the function knows which row comes 1st, 2nd, 3rd, or prior/next.
Crucial for ranking (ROW_NUMBER), running totals, and time offsets (LAG / LEAD).

3. Window Frames: ROWS vs RANGE

When you include ORDER BY inside OVER() , PostgreSQL creates a **sliding frame** (a moving subset of rows inside the partition).

UNBOUNDED PRECEDING	(First row of partition)	
1 PRECEDING	(Row immediately before)	
CURRENT ROW	(The row currently computed)	← We are here
1 FOLLOWING	(Row immediately after)	
UNBOUNDED FOLLOWING	(Last row of partition)	

The Difference between ROWS and RANGE :

- ROWS** : Counts **physical rows**. (e.g., ROWS BETWEEN 1 PRECEDING AND CURRENT ROW = exactly 2 rows).
- RANGE** : Evaluates **values**. If two rows have the same value in the ORDER BY column (ties), RANGE lumps them together as peers.

⚠ The Default Frame Trap (Must-Know!)

Whenever you write ORDER BY inside OVER() , PostgreSQL automatically applies this hidden default frame:

RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

- For running sums, this means from the start of the partition up to the current row.
- Danger with LAST_VALUE()** : Because the default frame stops at CURRENT ROW , LAST_VALUE() will just return the *current row's value*, not the true last row of the partition! (We'll see how to fix this below).

4. Category 1: Ranking Functions

Ranking functions assign an integer rank to each row based on an ordering.

Sample Data for Ranking:

Let's look at students and their exam scores:

student_name	subject	score
Alice	Math	95

student_name	subject	score
Bob	Math	90
Charlie	Math	90
David	Math	80

1. ROW_NUMBER()

Assigns a **strict, unique sequential integer** starting from 1 to every row in the partition. It **never gives duplicate numbers**, even if values tie.

Syntax:

```
ROW_NUMBER() OVER (  
    [PARTITION BY partition_col]  
    ORDER BY sort_col [ASC|DESC]  
)
```

Example:

```
SELECT  
    student_name, subject, score,  
    ROW_NUMBER() OVER (PARTITION BY subject ORDER BY score DESC) AS rn  
FROM exams;
```

Output:

student_name	score	rn	Explanation
Alice	95	1	Highest score
Bob	90	2	Tied with Charlie, but gets 2
Charlie	90	3	Tied with Bob, but gets 3
David	80	4	Next score

When to use ROW_NUMBER() :

- **Deduplication & Finding Latest Record:** e.g., "Find the latest order per customer" (WHERE rn = 1).
- **Pagination:** e.g., rows 11 to 20.

2. RANK()

Assigns ranks based on the ORDER BY column. **If rows tie, they receive the SAME rank, but the next rank SKIPS numbers (leaves gaps).**

Syntax:

```
RANK() OVER (  
    [PARTITION BY partition_col]  
    ORDER BY sort_col [ASC|DESC]  
)
```

Example:

```
SELECT  
    student_name, subject, score,  
    RANK() OVER (PARTITION BY subject ORDER BY score DESC) AS rnk  
FROM exams;
```

Output:

student_name	score	rnk	Explanation
Alice	95	1	1st Place
Bob	90	2	Tied for 2nd Place
Charlie	90	2	Tied for 2nd Place
David	80	4	Rank 3 is skipped! Because 2 people tied at rank 2.

When to use RANK() :

- True athletic/competition leaderboards (e.g., Olympic medals where two silver medalists mean no bronze is awarded).

3. DENSE_RANK()

Assigns ranks based on the ORDER BY column. **If rows tie, they receive the SAME rank, but the next rank DOES NOT SKIP numbers (no gaps).**

Syntax:

```
DENSE_RANK() OVER (  
    [PARTITION BY partition_col]  
    ORDER BY sort_col [ASC|DESC]  
)
```

Example:

```
SELECT  
    student_name, subject, score,  
    DENSE_RANK() OVER (PARTITION BY subject ORDER BY score DESC) AS dense_rnk  
FROM exams;
```

Output:

student_name	score	dense_rnk	Explanation
Alice	95	1	1st Place
Bob	90	2	Tied for 2nd Place
Charlie	90	2	Tied for 2nd Place
David	80	3	No gap! Next rank is 3.

When to use DENSE_RANK() :

- "Find the top 3 highest salaries in each department." Even if multiple employees earn the top salary, `DENSE_RANK() <= 3` captures the top 3 distinct pay levels.

 Ranking Comparison Side-by-Side:

score	ROW_NUMBER()	RANK()	DENSE_RANK()
100	1	1	1
90	2	2	2
90	3	2 (tie)	2 (tie)
80	4	4 (skipped 3)	3 (no skip)
70	5	5	4

5. Category 2: Distribution Functions

Distribution functions calculate relative percentiles and bucket splits across a dataset.

4. PERCENT_RANK()

Calculates the **relative rank** (percentile rank) of a row on a scale from **0.0 to 1.0**.

Formula:

$$\text{PERCENT_RANK} = \frac{\text{RANK} - 1}{\text{Total Rows in Partition} - 1}$$

- The highest/first row is always `0.0`.
- The lowest/last row is always `1.0`.

Example:

```
SELECT
  student_name, score,
  ROUND(PERCENT_RANK() OVER (ORDER BY score ASC)::numeric, 2) AS pct_rank
FROM exams;
```

When to use PERCENT_RANK() :

- Standardized test scores (e.g., "This student scored in the 90th percentile").

5. CUME_DIST()

Calculates the **Cumulative Distribution**: the proportion of rows whose values are **less than or equal to** the current row's value.

Formula:

$$\text{CUME_DIST} = \frac{\text{Count of rows with value } \leq \text{Current row's value}}{\text{Total Rows in Partition}}$$

- The value ranges strictly from > 0.0 to 1.0 .

Example:

```
SELECT
    student_name, score,
    ROUND(CUME_DIST() OVER (ORDER BY score ASC)::numeric, 2) AS cum_dist
FROM exams;
```

When to use CUME_DIST() :

- Risk modeling and SLAs (e.g., "What percentage of orders are delivered within 48 hours?").

6. NTILE(n)

Divides an ordered partition as evenly as possible into **n numbered buckets** (from 1 to **n**).

Syntax:

```
NTILE(num_buckets) OVER (
    [PARTITION BY partition_col]
    ORDER BY sort_col [ASC|DESC]
)
```

Example: Customer Spending Quartiles (**n = 4**)

If you have 8 customers ordered by total spend:

```
SELECT
    customer_id, total_spend,
    NTILE(4) OVER (ORDER BY total_spend DESC) AS quartile
FROM customer_summary;
```

Output:

customer_id	total_spend	quartile	Meaning
Cust_1	\$5,000	1	Top 25% VIP Spenders
Cust_2	\$4,200	1	Top 25% VIP Spenders
Cust_3	\$3,100	2	High Spenders
Cust_4	\$2,500	2	High Spenders
Cust_5	\$1,800	3	Moderate Spenders
Cust_6	\$1,200	3	Moderate Spenders
Cust_7	\$600	4	Low Spenders
Cust_8	\$200	4	Low Spenders

When to use NTILE() :

- Customer segmentation (Quartiles `NTILE(4)` , Deciles `NTILE(10)` , Percentiles `NTILE(100)`).
- Dividing workload evenly across `n` parallel pipeline workers.

6. Category 3: Navigation / Offset Functions

Navigation functions allow you to look **backward** or **forward** across rows without writing complex self-joins on dates or IDs.

7. LAG()

Fetches a column value from a row `offset` **positions BEFORE (above)** the current row in the partition.

Syntax:

```
LAG(column_to_fetch [, offset_rows [, default_value]]) OVER (  
  [PARTITION BY partition_col]  
  ORDER BY sort_col [ASC|DESC]  
)
```

- `offset_rows` : Number of rows to look back (Default is `1`).
- `default_value` : Value to return if there is no previous row (Default is `NULL`).

Real-World Example: Month-over-Month (MoM) Revenue Growth

```
SELECT  
  month,  
  revenue,  
  LAG(revenue, 1, 0) OVER (ORDER BY month) AS prev_month_revenue,  
  revenue - LAG(revenue, 1, 0) OVER (ORDER BY month) AS mom_growth  
FROM monthly_sales;
```

Output:

month	revenue	prev_month_revenue	mom_growth
2026-01	\$10,000	\$0 (default)	+\$10,000
2026-02	\$14,000	\$10,000	+\$4,000
2026-03	\$12,000	\$14,000	-\$2,000

When to use LAG() :

- MoM / YoY financial growth metrics.
- Calculating elapsed time between events (e.g., Days since customer's previous order).

8. LEAD()

Fetches a column value from a row **offset positions AFTER (below)** the current row in the partition.

Syntax:

```
LEAD(column_to_fetch [, offset_rows [, default_value]]) OVER (  
  [PARTITION BY partition_col]  
  ORDER BY sort_col [ASC|DESC]  
)
```

Real-World Example: Tracking Shipment Status Lifecycle

```
SELECT  
  shipment_id,  
  event_type,  
  event_timestamp,  
  LEAD(event_timestamp) OVER (  
    PARTITION BY shipment_id  
    ORDER BY event_timestamp  
  ) AS next_event_timestamp  
FROM logistics.delivery_events;
```

Output:

shipment_id	event_type	event_timestamp	next_event_timestamp
SHP-01	Created	10:00 AM	10:30 AM
SHP-01	Packed	10:30 AM	12:00 PM
SHP-01	Delivered	12:00 PM	NULL (last event)

When to use LEAD() :

- Sessionization & duration analysis (duration of current step = next_event_timestamp - event_timestamp).

7. Category 4: Value Functions

Value functions grab the first, last, or N-th value from the window frame.

9. FIRST_VALUE()

Returns the value from the **very first row** of the window frame.

Syntax:

```
FIRST_VALUE(column) OVER (  
    [PARTITION BY partition_col]  
    ORDER BY sort_col [ASC|DESC]  
    [frame_clause]  
)
```

Example: Compare Every Product Price with the Cheapest Product in its Category

```
SELECT  
    category_id,  
    product_name,  
    cost_price,  
    FIRST_VALUE(product_name) OVER (  
        PARTITION BY category_id  
        ORDER BY cost_price ASC  
    ) AS cheapest_product_in_category  
FROM catalog.products;
```

10. LAST_VALUE()

Returns the value from the **last row** of the window frame.

⚠ THE CRITICAL TRAP:

If you just write `ORDER BY ...`, the default frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.
The window frame stops at the **current row**, so `LAST_VALUE()` will return the **current row**, not the true last row!

The Correct Way (Explicit Frame):

To make `LAST_VALUE()` see the entire partition, you **must** specify `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`:

```
--  CORRECT LAST_VALUE USAGE
SELECT
  category_id,
  product_name,
  cost_price,
  LAST_VALUE(product_name) OVER (
    PARTITION BY category_id
    ORDER BY cost_price ASC
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS most_expensive_product_in_category
FROM catalog.products;
```

11. NTH_VALUE()

Returns the value from the *N*-th row of the window frame (or `NULL` if no such row exists).

Syntax:

```
NTH_VALUE(column, n) OVER (
  [PARTITION BY partition_col]
  ORDER BY sort_col [ASC|DESC]
  [frame_clause]
)
```

Example: Find the 2nd Most Expensive Product in Each Category

```
SELECT
  category_id,
  product_name,
  cost_price,
  NTH_VALUE(product_name, 2) OVER (
    PARTITION BY category_id
    ORDER BY cost_price DESC
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS second_highest_product
FROM catalog.products;
```

8. Category 5: Common Aggregate-as-Window Functions

Any standard SQL aggregate function (`SUM` , `AVG` , `COUNT` , `MIN` , `MAX`) automatically becomes a window function when you attach an `OVER()` clause to it!

1. Cumulative Running Total

```
SELECT
    order_timestamp::date AS order_date,
    total_amount,
    -- Running total across time
    SUM(total_amount) OVER (
        ORDER BY order_timestamp
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total_revenue
FROM orders.orders;
```

2. 7-Day Moving Average

```
SELECT
    order_date,
    daily_revenue,
    -- 7-day moving average (Current day + previous 6 days)
    AVG(daily_revenue) OVER (
        ORDER BY order_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS moving_avg_7d
FROM orders.v_daily_revenue;
```

3. Ratio to Total / Percentage Contribution

```
SELECT
    category_id,
    product_name,
    cost_price,
    -- Total cost of all products in this category
    SUM(cost_price) OVER (PARTITION BY category_id) AS category_total_cost,
    -- Percentage of category total
    ROUND((cost_price / SUM(cost_price) OVER (PARTITION BY category_id)) * 100, 2) AS pct_of_category
FROM catalog.products;
```

9. SQL Execution Order

Why does this query produce a syntax error?

```
-- ❌ SYNTAX ERROR: Window functions not allowed in WHERE
SELECT customer_id, order_id,
    ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_timestamp DESC) AS rn
FROM orders.orders
WHERE ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_timestamp DESC) = 1;
```

The SQL Engine Order of Operations:

1.

FROM / JOIN

(Find tables & join them)

2.

WHERE

(Filter raw input rows)

3.

GROUP BY

(Collapse into groups)

4.

HAVING

(Filter collapsed groups)

5.

WINDOW FUNCTIONS

(Compute OVER clauses)

6.

SELECT

(Select output columns)

7.

DISTINCT

(Remove duplicate rows)

8.

ORDER BY

(Final sort)

9.

LIMIT / OFFSET

(Paginate output)

Because `WHERE` runs at **Step 2** and Window Functions are evaluated at **Step 5**, the window calculations **do not exist yet** when `WHERE` is filtering!

The Standard Pattern: CTE or Subquery

To filter by a window function result, wrap it in a CTE:

```
--  CORRECT PATTERN
WITH ranked_orders AS (
  SELECT
    customer_id,
    order_id,
    order_timestamp,
    total_amount,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id
      ORDER BY order_timestamp DESC
    ) AS rn
  FROM orders.orders
)
SELECT customer_id, order_id, order_timestamp, total_amount
FROM ranked_orders
WHERE rn = 1;
```

10. Master Cheat Sheet

#	Function	Category	Purpose	Typical Real-World Use Case
1	<code>ROW_NUMBER()</code>	Ranking	Unique sequential numbering (1, 2, 3, 4).	Deduplication, fetching latest/first record per user.
2	<code>RANK()</code>	Ranking	Rank with gaps on ties (1, 1, 3, 4).	Competition leaderboards.
3	<code>DENSE_RANK()</code>	Ranking	Rank without gaps on ties (1, 1, 2, 3).	Finding Top N salary/spending tiers without skipping ranks.
4	<code>PERCENT_RANK()</code>	Distribution	Relative rank between 0.0 and 1.0.	Percentile scoring.
5	<code>CUME_DIST()</code>	Distribution	Cumulative distribution ($\leq$ current value/total).	SLA compliance, distribution curves.

#	Function	Category	Purpose	Typical Real-World Use Case
6	NTILE(<i>n</i>)	Distribution	Split partition into <i>n</i> equal buckets.	Customer quartiles (<i>n</i> = 4), deciles (<i>n</i> = 10).
7	LAG()	Navigation	Read value from prior row (<i>n</i> rows back).	Month-over-Month growth, time between events.
8	LEAD()	Navigation	Read value from next row (<i>n</i> rows ahead).	Step duration in user funnels.
9	FIRST_VALUE()	Value	Get first value in frame.	Comparing current item to category minimum.
10	LAST_VALUE()	Value	Get last value in frame (<i>requires explicit frame</i>).	Comparing current item to category maximum.
11	NTH_VALUE()	Value	Get <i>N</i> -th value in frame.	Finding runner-up / 2nd highest item.
12	SUM()/AVG() OVER()	Aggregate	Running or partition-level aggregates.	Cumulative revenue, 7-day rolling averages.